Regex et réécriture d'URL
Apache, .htaccess et logique pas à pas

Support de cours premium — GRETA
Linux / Apache / Regex / SEO / URL Rewriting

Objectif global du cours

Comprendre d'abord les expressions régulières (regex), puis utiliser cette logique pour réécrire des URLs dans Apache afin de transformer une URL technique en URL plus propre, plus lisible et mieux exploitable pour le référencement.

Le transcript brut a été nettoyé, remis dans l'ordre, puis transformé en support pédagogique détaillé. Le document garde l'esprit du cours, mais il corrige les imprécisions, clarifie les exemples et ajoute le pourquoi de chaque étape.

1. Vision d'ensemble du module

Le professeur enchaîne ici deux idées qui sont liées :

- les regex, qui servent à reconnaître des motifs dans une chaîne de caractères ;
- la réécriture d'URL, qui utilise justement ce type de logique pour reconnaître une URL d'entrée et la transformer vers une URL cible.

Autrement dit : on apprend les regex pour ensuite comprendre comment Apache peut transformer une adresse comme `film-7.html` en appel réel vers un script PHP.

Exemple central du cours

```
URL réelle
http://www.cinema.greta/fiche.php?iddvd=1

URL réécrite
http://www.cinema.greta/film-1.html
```

2. Pourquoi réécrire une URL ?

Le transcript insiste fortement sur un point : une URL avec variables est pratique techniquement, mais moins bonne visuellement et souvent moins favorable pour le référencement.

Type d'URL	Exemple	Lecture
URL technique	fiche.php?iddvd=7	Le script comprend facilement, mais l'URL expose la mécanique interne.
URL réécrite	film-7.html	Plus simple à lire, plus propre, plus proche d'une page web « normale ».

Idée clé du prof

Le moteur de recherche préfère des URLs propres, stables et lisibles. En pratique, presque tous les sites modernes cachent leurs paramètres techniques derrière une réécriture d'URL.

3. Les regex : définition simple

Une regex est un motif de recherche. Elle ne cherche pas forcément une phrase exacte : elle peut chercher un type de mot, un type de structure, un nombre, une séquence, une fin de chaîne, un début de chaîne, etc.

Le professeur rappelle d'ailleurs que `grep` repose déjà sur ce principe : le mot « grep » vient historiquement de la logique des regular expressions.

Exemple de rappel donné dans le cours

```
grep http /etc/services
```

Convention utilisée dans le cours

Dans les explications du prof, les regex sont souvent encadrées par des `# ... #` pour bien les distinguer visuellement du reste de la phrase.

4. Premier niveau : recherche simple et sensibilité à la casse

On commence par le plus simple : chercher un mot dans une chaîne.

Ligne	Chaîne	Regex	Résultat
1	J'aime linux	#linux#	vrai
2	J'aime linux	#Linux#	faux
3	J'aime linux	#Linux#i	vrai

Lecture pédagogique :

- `#linux#` trouve `linux` car la chaîne contient bien ce mot.
- `#Linux#` échoue car la regex cherche un L majuscule, alors que la chaîne a un l minuscule.
- L'option `i` rend la recherche insensible à la casse.

À retenir

Sans option `i`, la casse compte. Avec `i`, `linux`, `Linux`, `LINUX` ou `LiNuX` sont traités comme équivalents.

5. Le pipe | : chercher « ceci OU cela »

Le caractère `|` sert à exprimer une alternative.

Ligne	Chaîne	Regex	Résultat
4	J'aime linux	#linux|microsoft#	vrai

- Ici, la regex demande : trouve `linux` ou `microsoft`.
- La chaîne contient `linux`, donc le résultat est vrai.

Traduction mentale

`|` se lit « ou ».

6. Le début ^ et la fin $ de chaîne

Le professeur insiste sur deux symboles fondamentaux : `^` pour le début, `$` pour la fin.

Ligne	Chaîne	Regex	Résultat
5	J'aime linux	#^linux#	faux
6	J'aime linux	#linux$#	vrai
7	J'aime linux.	#linux$#	faux
8	J'aime linux	#^linux$#	faux

- `^linux` veut dire : la chaîne doit commencer par `linux`.
- `linux$` veut dire : la chaîne doit finir par `linux`.
- `^linux$` veut dire : la chaîne entière doit être exactement `linux`.

Détail important

Dans `J'aime linux.`, le point final empêche `linux$` d'être vrai, car la chaîne se termine en réalité par `linux.`

7. Les crochets [] : OU sur des lettres

Le professeur distingue bien deux usages :

- `|` pour faire une alternative entre mots ou blocs entiers ;
- `[]` pour exprimer un choix entre caractères.

Ligne	Chaîne	Regex	Résultat
9	la nuit les chats sont gris	#gris|gros|gras#	vrai
10	la nuit les chats sont gris	#gr[iao]s#	vrai
11	les chats sont gris	#aj#	faux
12	les chats sont gris	#[aj]#	vrai

`gr[iao]s` veut dire :

- commence par `gr` ;
- puis accepte soit `i`, soit `a`, soit `o` ;
- puis termine par `s`.

Cela permet donc de trouver `gris`, `gras` ou `gros`.

8. Intervalles dans les crochets

Ligne	Chaîne	Regex	Résultat
13	les chats sont gris	#[a-j]#	vrai
14	les chats sont gris	#[a-jA-J]#	vrai
15	les chats sont gris	#[a-j]#i	vrai
16	Nous sommes en 2026	#[0-9]#	vrai
17	Nous sommes en 2026	#[7-9]#	faux

- `[a-j]` : une lettre entre a et j.
- `[a-jA-J]` : une lettre entre a et j, en minuscule ou majuscule.
- `[0-9]` : un chiffre.
- `[7-9]` : un chiffre entre 7 et 9.

Raccourci mental

Le tiret dans `[]` exprime un intervalle.

9. Quantificateurs : +, ?, *

Ces symboles disent combien de fois un caractère ou un motif peut apparaître.

Symbole	Sens	Lecture simple
+	1 fois ou plus	Le motif doit exister au moins une fois.

?	0 ou 1 fois	Le motif est optionnel, mais ne peut pas apparaître deux fois.
*	0 fois ou plus	Le motif peut être absent, ou répété autant qu'on veut.

Ligne	Chaîne	Regex	Résultat
18	il fait froid ce matin	#a|aa|aaa|aaaa#	vrai
19	il fait froid ce matin	#a+#	vrai
20	les chats sont gris	#z+#	faux
21	les chats sont gris	#z*#	vrai
22	je passe la porte	#s?#	faux

Pourquoi la ligne 21 est vraie ?

- Parce que `z*` autorise zéro occurrence.
- Même s'il n'y a pas de z, la regex peut tout de même être satisfaite.

Piège fréquent

L'étoile `*` donne souvent vrai très facilement, parce qu'elle accepte aussi le cas « zéro occurrence ».

10. Les accolades { } : écrire des quantités précises

Les accolades permettent d'écrire explicitement une quantité minimale et maximale.

Écriture	Équivalent	Sens
a{1,}	a+	au moins 1 fois
a{0,1}	a?	0 ou 1 fois

a{0,}	a*	0 fois ou plus
a{1,4}	a|aa|aaa|aaaa	de 1 à 4 fois

Ligne	Chaîne	Regex	Résultat
23	nous sommes en 2026	#[0-9]{5,5}#	faux
24	nous sommes en 2026	#[0-9]{4,5}#	vrai

- `[0-9]{5,5}` cherche exactement 5 chiffres qui se suivent.
- `[0-9]{4,5}` cherche 4 ou 5 chiffres qui se suivent.

Comme `2026` contient 4 chiffres consécutifs, la ligne 24 est vraie.

11. Le backslash \ : annuler le sens spécial d'un caractère

Le professeur donne l'exemple du signe dollar. En regex, `$` signifie normalement fin de chaîne. Si on veut chercher le caractère dollar lui-même, il faut l'échapper.

Ligne	Chaîne	Regex	Résultat
25	il me reste 10 $ sur mon compte	#\$#	vrai

Lecture simple

Le backslash `\` dit à la regex : « n'interprète plus ce symbole comme une instruction, traite-le comme un caractère normal ».

12. Regex avec caractère optionnel

Ligne	Chaîne	Regex	Résultat
26	je suis dans les bois	#bor?is#	vrai

Le `r?` signifie que la lettre `r` peut être absente ou présente une seule fois. Le motif peut donc reconnaître une forme avec ou sans cette lettre optionnelle.

13. Deux exemples plus avancés donnés en fin de cours

Ligne	Chaîne	Regex	Résultat
28	Google est mon ami	#^go+$#i	faux
29	Google est mon ami	#[a-zA-Z]o{3,5}#	faux
30	01020304	#^0([0-9]{2}){2,3}#	vrai
31	01020304	#^0([0-9]{2}){2,3}$#	faux

Le dernier exemple est très pédagogique. On peut le lire ainsi :

- `^0` : la chaîne doit commencer par 0 ;
- `([0-9]{2})` : puis il faut un bloc de 2 chiffres ;
- `{2,3}` : ce bloc doit être répété 2 ou 3 fois.

Donc on attend une structure du type `0 + 2 chiffres + 2 chiffres` ou `0 + 2 chiffres + 2 chiffres + 2 chiffres`.

Pourquoi la ligne 31 devient fausse

Avec le `$`, on impose que la chaîne entière respecte exactement la structure jusqu'à la fin. Le moindre caractère en trop ou la mauvaise longueur fait échouer le motif.

14. Exercice : construire une regex pour un email

Le professeur termine avec un mini-cas pratique : écrire une regex pour valider un email selon des règles données.

Règle	Interprétation
Il doit y avoir un @	L'adresse doit contenir une séparation entre partie locale et domaine.

Avant @	Le premier caractère doit être une lettre ou un chiffre.
Avant @	Les caractères autorisés sont les lettres, chiffres, point et underscore.
Avant @	Minimum 2 caractères.
Après @	Doit commencer par une lettre ou un chiffre.
Après @	On doit trouver un point `.`.
Après le point	Uniquement des lettres, minimum 2.

Exemples fournis dans le cours :

Adresse	**Résultat attendu**
alex.terrieur@hotmail.com	ok
alex-terrieur@hotmail.com	pas ok
alex.terrieur@00hotmail.com	ok
alex.terrieur@hotmail75.com	ok
alex.terrieur@hotmail.c0m	pas ok

Deux propositions données en cours

```
Regex Jordan
^[a-zA-Z][a-zA-Z0-9._]*@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$

Regex Pascal
^[a-z0-9][a-z0-9._]{1,}@[a-z0-9][a-z0-9_]{1,}.[a-z]{2,}$
```

La première proposition est plus stricte sur certains points et plus propre pour l'usage visé ici, notamment grâce à l'échappement clair du point avant l'extension.

15. Passage vers Apache : pourquoi les regex sont utiles pour la réécriture d'URL

Une fois la logique regex comprise, le professeur la reconnecte à Apache. L'idée est simple :

- Apache reçoit une URL propre comme `film-1.html` ;
- il vérifie si cette URL correspond à un motif ;
- si oui, il extrait les variables ;
- puis il appelle l'URL réelle correspondante.

Objectif Apache

```
URL réelle
http://www.cinema.greta/fiche.php?iddvd=1

URL réécrite
http://www.cinema.greta/film-1.html
```

16. Activer la réécriture dans Apache

Étape 1

```
sudo a2enmod rewrite
```

Il faut ensuite autoriser `.htaccess` dans la configuration Apache du site concerné.

Bloc à placer dans apache2.conf

```
<Directory /home/client4/>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
</Directory>
```

Appliquer la modification

```
sudo systemctl reload apache2
```

Pourquoi `AllowOverride All` ?

Parce que la réécriture d'URL sera écrite dans `.htaccess`. Si Apache n'a pas le droit d'utiliser ce fichier,

les règles ne serviront à rien.

17. Première règle .htaccess : une variable iddvd

.htaccess version simple

```
RewriteEngine on
# url réelle   => http://www.cinema.greta/fiche.php?iddvd=5
# url réécrite => http://www.cinema.greta/film-5.html
RewriteRule ^film-([1-9][0-9]{0,}).html$ fiche.php?iddvd=$1
```

Élément	Rôle
RewriteEngine on	Active le moteur de réécriture.
^film-	L'URL doit commencer par `film-`.
([1-9][0-9]{0,})	Capture un nombre entier positif.
.html$	L'URL doit finir par `.html`.
fiche.php?iddvd=$1	La variable capturée devient la valeur de `iddvd`.

18. Deuxième règle .htaccess : langue + identifiant

Le professeur enrichit ensuite la logique pour faire passer deux variables dans l'URL : une langue et un identifiant de film.

.htaccess version avancée

```
RewriteEngine on
# url réelle   => http://www.cinema.greta/fiche.php?iddvd=5&lg=fr
# url réécrite => http://www.cinema.greta/film-fr-5.html
RewriteRule ^film-([a-z]{2,})-([1-9][0-9]{0,}).html$ fiche.php?iddvd=$2&lg=$1
```

- La première parenthèse capture la langue → `$1`.
- La deuxième parenthèse capture l'ID → `$2`.
- Apache reconstruit ensuite l'URL réelle avec les bons paramètres.

Test final demandé

```
Test navigateur :
http://www.cinema.greta/film-uk-7.html
```

19. Lien avec le code PHP

Le cours fournit ensuite les fichiers `fiche.php` et `index.php`.

Variables récupérées dans fiche.php

```
$numCom = $_GET['iddvd'];
$language = $_GET['lg'];
```

Cela montre très bien le lien entre Apache et PHP :

- Apache capture les morceaux utiles dans l'URL réécrite ;
- Apache les remet en paramètres GET ;
- PHP lit ensuite ces paramètres avec `$_GET`.

Affichage de la langue dans fiche.php

```
echo "<h1 align=center>$titre $language</h1>";
```

Dans `index.php`, les liens d'origine sont encore écrits avec l'URL réelle. Le but du TP est précisément de les faire évoluer vers une URL propre.

20. Learning by doing — déroulé pas à pas

Étape	Action concrète	But pédagogique
1	Comprendre ce qu'est une regex	Savoir reconnaître un motif dans une chaîne.
2	Comprendre `i`, `|`, `^`, `$`, `[]`	Maîtriser les bases.
3	Comprendre `+`, `?`, `*`, `{}`	Maîtriser les quantités.
4	Comprendre `\`	Savoir chercher un caractère

		spécial.
5	Construire une regex email	Passer d'exemples simples à une vraie règle.
6	Activer `mod_rewrite`	Préparer Apache.
7	Autoriser `.htaccess`	Permettre à Apache de lire les règles.
8	Écrire une RewriteRule simple	Faire la passerelle regex → Apache.
9	Ajouter une deuxième variable	Comprendre `$1`, `$2` et les groupes capturés.
10	Tester dans le navigateur	Valider que l'ensemble fonctionne.

Très bonne méthode de travail

Ne pas avancer trop vite. Faire une petite modification, tester, comprendre, puis seulement passer à la suivante.

21. Check-list de révision

- Je sais expliquer ce qu'est une regex.
- Je sais expliquer la différence entre recherche exacte et recherche insensible à la casse.
- Je sais dire à quoi sert `|`.
- Je sais utiliser `^` et `$`.
- Je sais lire un intervalle dans `[]`.
- Je sais expliquer `+`, `?`, `*`.
- Je sais réécrire `a+` sous forme `a{1,}`.
- Je sais expliquer le rôle du backslash `\`.
- Je sais comprendre une regex email simple.
- Je sais expliquer la différence entre URL réelle et URL réécrite.
- Je sais expliquer à quoi sert `.htaccess`.
- Je sais expliquer pourquoi `AllowOverride All` est indispensable.
- Je sais expliquer comment Apache transmet ensuite les variables à PHP via `$_GET`.

22. Conclusion

Ce cours fait le pont entre deux mondes :

- la logique abstraite des regex ;
- et leur usage concret dans Apache pour réécrire des URLs.

Si tu comprends bien les exemples du tableau, puis la règle `.htaccess`, tu possèdes déjà la logique de base utilisée dans énormément de projets web réels.